SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

StockSense is an AI-powered supply chain automation system designed to help businesses manage inventory efficiently by predicting demand, detecting stockout risks, and automating procurement workflows. The system connects inventory records stored in Google Sheets with an automation backend powered by n8n and integrates GLM-based reasoning to generate forecasting insights, supplier outreach emails, and weekly business reports. StockSense reduces manual work in inventory monitoring and supplier communication while improving supply chain decision-making.

1.1. Purpose:

This system analysis document highlights the technical scope, architectural design, and implementation decisions behind the development of StockSense. It explains how the system components interact, how data flows through the platform, and how AI reasoning is integrated as a service layer for demand forecasting and procurement automation.
So, the key elements of the system that have been covered are as followed:

1. ***Architecture**:*
   StockSense is built on a web-based architecture where a Next.js frontend dashboard interacts with an n8n automation backend through webhook APIs. The backend connects to external services such as Google Sheets, Gmail API, SerpApi, and the GLM model API. Each workflow module is separated into independent automation pipelines to support scalability and maintainability.
2. ***Data Flows:***
   Inventory data is stored inside Google Sheets and fetched by n8n workflows. The workflows clean and format the data, then forward it to the GLM API for forecasting and decision reasoning. Results are written back into Google Sheets for persistence and are also used to trigger procurement email generation and vendor discovery operations.
3. ***Model Process:***
   The system model represents the end-to-end automation process: inventory data ingestion, forecasting, stock risk classification, supplier matching, supplier email drafting, email sending, and weekly vendor discovery reporting. Each process is executed as a structured workflow inside n8n.
4. ***Role of Reference:***
   This document serves as a technical reference for developers, testers, and stakeholders to understand the overall system structure, design rationale, key workflows, and component interactions for StockSens

1.2. Background:

Businesses often manage inventory manually. This leads to overstocking or stock shortages. It also slows down supplier communication and reporting.

© UMHackathon 2026

1. *Previous Version:*
    a. The vast majority of businesses use spreadsheets or basic tools to handle inventory systems manually. They track stock but do not predict demand.
2. *Changes in Major Architectural Components*
    a. LLM reasoning using GLM 5
    b. Automation using n8n
3. *New capabilities that are only introduced in this*
    a. Predict demand before stock runs out
    b. Detect low stock automatically
    c. Generate supplier message
    d. Generate weekly reports

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Supply Manager	*Monitors inventory and approves procurement strategies.*	*A clean dashboard and automated weekly discovery newsletters.*
Procurement Team	*Manages vendor relationships and order fulfillment.*	*Automated drafting of urgent restock emails with accurate product data.*
Suppliers	*Receives and fulfills restock orders.*	*Clear, structured order requests with lead times and payment terms.*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validating the system behavior for all the user roles*	*Defined scope, the request and response formats, and coverage of edge cases*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Dashboard + Automation Workflows
Architecture	*Next.js Frontend + n8n Orchestration Backend + GLM5 AI Service Layer*

StockSense is structured as a client-server automation platform. The frontend dashboard is built using Next.js and serves as the primary user interface for supply chain managers. The backend consists of n8n workflows which orchestrate inventory forecasting, supplier communication, and supplier discovery. Google Sheets is used as the persistent storage layer for inventory data, supplier lists, forecasts, and communication logs. The system integrates the GLM model API for reasoning and content generation, Gmail API for automated supplier outreach, and SerpApi for discovering new vendors online.

2.1.2 LLM as Service Layer

StockSense integrates the GLM model API as an independent reasoning service layer rather than treating AI as a generic component. The system builds structured prompts using cleaned inventory data and supplier context, sends these prompts to GLM, receives structured JSON responses, and parses the results for further automation. Token control is enforced through batching and chunking strategies to ensure requests remain within model limits.

Dependency Diagram

- Next.js dashboard triggers n8n workflows through webhook endpoints
- n8n pulls inventory data from Google Sheets
- n8n constructs structured prompts and sends them to GLM API
- GLM returns JSON output including stockout risk, urgency score, and recommendations
- n8n parses output and updates Google Sheets
- n8n triggers Gmail API to send procurement emails

© UMHackathon 2026

- n8n triggers SerpApi for vendor discovery queries
- n8n generates and emails weekly reports

Prompt Construction

Prompt templates are created inside n8n nodes, where inventory data is formatted into structured JSON instructions such as:

- stock levels
- weekly sales rate
- supplier lead time
- reorder threshold

Context Window Content

The context window includes:

- inventory history per product
- region-level demand patterns
- supplier email records
- past reorder logs and timestamps

Response Parsing & Handoff

GLM outputs structured JSON containing:

- predicted demand quantity
- stockout probability score
- urgency classification (low/medium/high)
- recommended reorder quantity.
- n8n parses this output and forwards the extracted fields to the procurement and reporting workflows.

© UMHackathon 2026

Token Limitation Handling

Large inventory datasets are split into batches by product category or region. Each batch is processed separately before being merged back into the master report.

Sequence Diagram

Example Flow: Weekly Forecast + Procurement Run

1. Scheduler triggers the Master Orchestrator workflow weekly
2. n8n fetches inventory dataset from Google Sheets
3. Inventory data is cleaned and categorized into batches
4. Each batch is sent to GLM API for forecasting
5. GLM returns stockout risk predictions and reorder recommendations
6. n8n writes forecast results into Google Sheets
7. Procurement workflow checks urgent products
8. Supplier matching is performed using supplier database sheet
9. GLM drafts procurement email messages
10. Gmail API sends emails to suppliers
11. Communication logs are updated in Google Sheets
12. Weekly summary report is generated and emailed to supply manager

2.2. Technological Stack

Layer	***Technology***
Frontend	*Next.js + TypeScript + Tailwind CSS*
Backend	*n8n Workflow Automation*
Database	*Google Sheets*

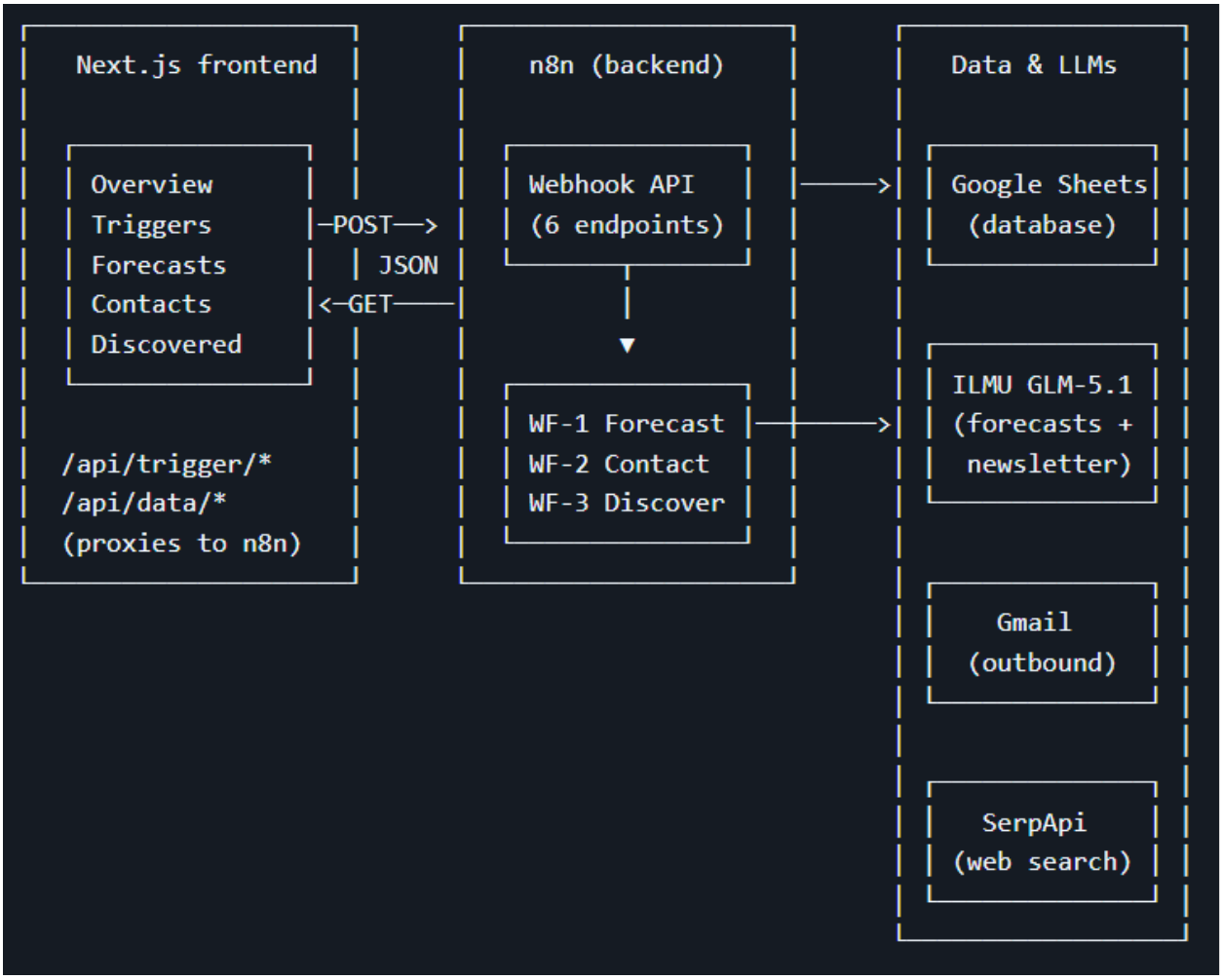
© UMHackathon 2026

AI / Reasoning	*GLM API*
External Search	*SerpApi*
Email Communication	*Gmail API*
Deployment	*Docker (n8n), Node.js (Next.js frontend)*

2.3. Key Data Flows

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

2.4. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.5. Normalized Database Schema

Based on the data in your spreadsheet, I have designed a normalized database schema. This schema eliminates redundancy (like repeating category names or company details) and establishes clear relationships between products, suppliers, inventory, and forecasts.

1. Core Master Data

These tables store the fundamental entities of your business.

- **`Categories`**
  - `category_id` (Primary Key)
  - `category_name` (e.g., Electronics, Clothing, Toys)
- **`Products`**
  - `product_id` (Primary Key)
  - `category_id` (Foreign Key to Categories)
- **`Stores`**
  - `store_id` (Primary Key)
  - `region` (e.g., North, South)
- **`Suppliers`**
  - `supplier_id` (Primary Key)
  - `company_name`
  - `contact_email`
  - `contact_phone`
  - `city`
  - `country`
  - `reliability_score`
  - `status` (Active/Inactive)

2. Relationships & Pricing

This table manages the link between suppliers and the specific products they provide.

- **`Supplier_Products`**
  - `supplier_id` (Foreign Key to Suppliers)
  - `product_id` (Foreign Key to Products)
  - `unit_price_myr`
  - `lead_time_days`
  - `minimum_order_qty`
  - `payment_terms`
  - `is_preferred` (Boolean)
  - `notes`
  - *Composite Primary Key: (`supplier_id`, `product_id`)*

3. Transactional & Historical Data

This table captures the daily snapshots of your inventory and sales performance.

- **`Stock_History`**
  - `date`
  - `store_id` (Foreign Key to Stores)

© UMHackathon 2026

Email: umhackathon@um.edu.my

- `product_id` (Foreign Key to Products)
- `inventory_level`
- `units_sold`
- `units_ordered`
- `price`
- `discount`
- `weather_condition`
- `is_promotion` (Boolean)
- `competitor_pricing`
- `seasonality`
- `is_epidemic_impact` (Boolean)
- `actual_demand`
- *Composite Primary Key: (`date`, `store_id`, `product_id`)*

4. Forecasting & Intelligence

These tables track the results of your demand forecasting runs.

- **`Forecast_Runs`**
  - `run_id` (Primary Key)
  - `run_at` (Timestamp)
- **`Forecast_Results`**
  - `run_id` (Foreign Key to Forecast_Runs)
  - `product_id` (Foreign Key to Products)
  - `region`
  - `historical_avg_demand`
  - `historical_trend`
  - `predicted_demand`
  - `confidence_level`
  - `confidence_reason`
  - `key_drivers`
  - `stockout_risk`
  - `recommended_action`
  - `action_detail`

5. Communication & Discovery Logs

These tables track interactions with current and potential suppliers.

- **`Supplier_Contact_Logs`**
  - `log_id` (Primary Key)
  - `run_id` (Foreign Key to Forecast_Runs)
  - `supplier_id` (Foreign Key to Suppliers)
  - `product_id` (Foreign Key to Products)
  - `email_subject`
  - `email_priority`
  - `sent_to`
  - `status` (Sent/Failed)
- **`Supplier_Discovery_Logs`**
  - `discovery_id` (Primary Key)
  - `run_id` (Foreign Key to Forecast_Runs)
  - `discovered_at` (Timestamp)
  - `category_id` (Foreign Key to Categories)
  - `priority`
  - `company_name`

© UMHackathon 2026

Email: umhackathon@um.edu.my

- `primary_email`
- `website`
- `description`
- `status` (Discovered/Vetted)

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

	Features	**Description**
1	*Demand Forecasting*	*Batches regional inventory data from google sheets and uses GLM to predict stockouts and urgent reorder needs*
2	*Autonomous Procurement*	*Automatically matches high risk items to existing vendors and uses GLM to draft and send emails*
3	*Supplier Discovery*	*Leverages SerpApi to find new vendors for high risk categories and generates a weekly summary newsletter for the supply manager*

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	*The system must handle increasing volumes of inventory data without degrading performance during weekly runs*	*Use of n8n's batch processing to split large datasets into regional batches, allowing for parallel execution of workflows*
Reliability	*The system must prevent redundant communications to the same vendor.*	*Use a logic node in n8n that checks contact logs before triggering the Gmail API*
Maintainability	*The backend architecture must be modular to allow independent updates to different AI agents.*	*Separation of logic into different workflows: Master Orchestrator, Forecasting, Contacting, Discovery*

© UMHackathon 2026

Token Latency	*[GLM calls must respond within acceptable time for batch execution*	*Request chunking + timeout settings inside n8n*
Cost Efficiency	*Total token costs must be minimised by preventing the transmission of irrelevant metadata to the LLM*	*Implementation of slimming node to clean and truncate inventory data before sending it to the GLM*

3.3. Out of Scope / Future Enhancements

- Real-time forecasting with live POS integration
- Full procurement order placement system (purchase orders + invoices)
- Supplier contract management system
- Logistics and delivery route optimization
- Replacement of Google Sheets with enterprise database + ERP integration

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing:*** StockSense can adopt a staged rollout approach by enabling forecasting automation first for a limited set of products or regions. Once prediction accuracy is validated, supplier outreach automation can be enabled gradually. A/B testing can be used to compare different prompt templates for forecasting accuracy and supplier email effectiveness.

4.1.2. ***Emergency Rollback (ER) & Golden Release:*** In case of failures such as incorrect procurement emails or faulty predictions, workflows can be immediately disabled in n8n while reverting to the last stable workflow export. The Golden Release version is maintained as a backup workflow set to restore stable operations.

4.1.3. Priority Matrix:

P3 (Medium) • SerpApi vendor discovery fails • Weekly report generation delayed **Action:** Retry later, continue core forecasting	**P1 (Critical)** • Wrong supplier contacted • Duplicate spam emails sent **Action:** Immediate workflow shutdown + rollback
P4 (Low) • Dashboard UI bug • Minor formatting/report issues **Action:** Patch frontend, no system halt	**P2 (High)** • GLM API failure rate > 20% • Forecast results missing **Action:** Retry + fallback to rule-based alerts

© UMHackathon 2026

4.2. Monitoring:

4.2.1. Monitoring will be performed through n8n execution logs and Google Sheets logging. The system tracks workflow failures, API errors, and response delays. Alert triggers can be implemented by emailing the development team if workflows fail repeatedly or if API quotas are exceeded.

4.3. Assumptions:

- Businesses maintain updated inventory data inside Google Sheets
- Users have stable internet connection when accessing dashboard
- Supplier emails stored in database are valid
- GLM API availability remains stable during workflow execution

4.4. External Dependencies:

Tools	Purpose	Risks
GLM API	Core reasoning engine for forecasting + email/report generation	High risk if rate-limited or unavailable
Google Sheets API	Stores inventory, suppliers, forecasts, logs	Medium risk due to quota limitations
Gmail API	Sends procurement emails	Low risk but may trigger spam filters
SerpApi	Discovers new suppliers online	Medium risk due to inconsistent search quality
n8n	Workflow automation engine	Medium risk if server goes down

5. Project Management & Team Contributions

5.1. Project Timeline:

Day 1-3, Planning, brainstorming
Day 4-9, Project development,
Day 9-10, Documentation, Submission

5.2. Team Members Role:

Muhammad Ali: *System Architect, Backend Workflow Development (n8n), AI Prompt Engineering, Frontend Integration*

Eunhye Oh: *Documentation, UI content structuring, requirement analysis support*

© UMHackathon 2026

Recommendations:

- *Replace Google Sheets with PostgreSQL for better scalability and query reliability*
- *Introduce Redis caching for repeated GLM prompt results*
- *Add authentication and role-based access control for multiple business accounts*
- *Add alerting via Telegram/Slack for critical procurement failures*

© UMHackathon 2026

Email: umhackathon@um.edu.my